

Spécification — Framework d'apprentissage déterministe pour le jeu de dames international

Destinataire : Claude Code (agent d'implémentation autonome) **Projet parent** : Draught Master (`Ai-draught/`) **Module à construire** : `backend/pedagogy/` **Langage** : Python 3.11+ **Tests** : pytest **Stack existante à respecter** : FastAPI, SQLite (WAL), Scan (Hub v2), Pydantic, Anthropic SDK

0. Contexte et objectif

Le projet existant joue, analyse coup par coup avec Scan, et délègue l'explication en langage naturel à Claude via un prompt libre. Cette approche a deux limites :

1. **Hallucinations** : Claude peut inventer un motif tactique qui n'existe pas dans la position.
2. **Absence de progression structurée** : aucune capitalisation des forces/faiblesses du joueur.

Le module `pedagogy/` doit fournir une **couche d'explication déterministe** qui :

- Détecte des motifs tactiques et stratégiques connus du jeu de dames international (FMJD 10×10) à partir d'une position et de la variante principale Scan.
- Produit des verdicts pédagogiques associés à chaque coup d'une partie.
- Construit un profil agrégé de forces et faiblesses par utilisateur.
- Fournit à Claude un contexte structuré et fermé (liste de motifs détectés) que Claude ne peut qu'habiller en français, sans inventer de motif.

Principe directeur : toute affirmation pédagogique doit être traçable à un détecteur déterministe. Claude rédige, ne raisonne pas sur le damier.

1. Glossaire métier (jeu de dames international)

Avant tout code, l'agent doit avoir ce vocabulaire en tête. Toutes les constantes, noms de classes et noms de fonctions doivent l'utiliser.

Cases et géométrie

- **50 cases** numérotées 1 à 50, lecture de gauche à droite et de haut en bas, du point de vue des Noirs (la case 1 est en haut à gauche).
- **Rangées** : rangée 1 (cases 1–5), rangée 2 (6–10), ..., rangée 10 (46–50).
- **Centre** : cases 27, 28, 22, 23 (cœur), élargi à 16, 17, 18, 19, 21, 24, 26, 29, 31, 32, 33, 34 (centre élargi).
- **Ailes** : aile gauche cases 6, 16, 26, 36, 46 et adjacentes ; aile droite cases 15, 25, 35, 45 et adjacentes (du point de vue noir).
- **Grande diagonale** : 5–46 (de la case 5 à la case 46).
- **Rangée de promotion blancs** : 1–5. **Rangée de promotion noirs** : 46–50.

Pièces

- **Pion** (man) : avance d'une case en diagonale, peut prendre vers l'avant ET vers l'arrière.
- **Dame** (king) : se déplace de n'importe quel nombre de cases sur une diagonale libre, prend à distance.

Règles essentielles

- **Prise obligatoire** : si une prise est disponible, elle doit être jouée.
- **Prise maximale** : parmi les prises possibles, la séquence la plus longue (en nombre de pièces capturées) est obligatoire. Une dame compte autant qu'un pion pour le décompte.
- **Pièces capturées** restent sur le damier jusqu'à la fin de la séquence (règle dite "du soufflage" — les pièces ne sont retirées qu'à la fin).
- **Promotion** : un pion qui *finit son coup* sur la rangée adverse devient dame. Un pion qui traverse la rangée adverse pendant une rafle sans s'y arrêter ne promeut PAS.

Motifs tactiques nommés (FR/NL)

Liste de référence à implémenter dans cet ordre de priorité (P1 = MVP, P2 = v2, P3 = futur) :

Nom français	Nom anglais/NL	Priorité	Signature
Coup royal	Royal coup	P1	Rafle finale ≥ 7 pions au centre, axe 40→34→24→14→3 ou symétriques

Coup turc	Turkish stroke	P1	Une pièce capturée <i>deux fois</i> dans une même rafle (rafle en boucle)
Coup de talon	Heel stroke	P1	Rafle qui rentre dans une case puis ressort par la même case
Envoi à dame	Promotion sacrifice	P1	Sacrifice forcé menant à promotion en ≤ 2 demi-coups
Coup Philippe	Philippe stroke	P2	Rafle avec sacrifice initial sur l'aile
Coup Raphaël	—	P2	Sacrifice de pion 27 ou 24 amenant rafle 28x6 / 23x5
Coup de l'express	Express stroke	P2	Rafle longue (5+ pions) en ligne droite
Coup Bonnard	Bonnard stroke	P2	Sacrifice qui force promotion adverse puis dame capturée
Coup de mazette	Beginner stroke	P3	Combinaison élémentaire 2-3 pions, contexte ouverture
Coup Fabre	Fabre stroke	P3	Combinaison spécifique cf. Dubois ch.21

Concepts stratégiques

Concept	Métrique
Centre fort/faible	Nombre de pions propres dans le centre élargi
Pion arrière	Pion sur rangée de départ sans pion propre devant lui
Trou	Case vide dans la formation, atteignable par l'adversaire
Avant-poste	Pion avancé non capturable et soutenu
Enchaînement	Pion adverse cloué entre deux pions propres
Pion isolé	Pion sans pion propre adjacent en diagonale
Tempo	Différence de pions développés (rangée > 4 pour blancs, < 7 pour noirs)
Formation classique	Empilement 32-37-41 (blancs) ou 14-19-10 (noirs)

Formation Roozenburg	Empilement 28-32 + soutien 37 (blancs)
Percée	Pion à 2 coups de promotion sans piège possible
Opposition (finale)	Position où l'adversaire est en zugzwang

2. Architecture du module

```

backend/pedagogy/
├── __init__.py
├── types.py                # Dataclasses partagées
├── features/
│   ├── __init__.py
│   ├── material.py        # Comptes matériels
│   ├── geometry.py        # Centre, ailes, diagonales
│   ├── structure.py       # Trous, isolés, arrières, avant-postes
│   ├── mobility.py        # Coups légaux, prises menacées
│   └── formations.py      # Reconnaissance de formations connues
├── motifs/
│   ├── __init__.py
│   ├── base.py            # Classe abstraite MotifDetector
│   ├── coup_royal.py
│   ├── coup_turc.py
│   ├── coup_de_talon.py
│   ├── envoi_a_dame.py
│   ├── sacrifices.py
│   ├── prise_max_ratee.py
│   ├── enchainements.py
│   ├── percee.py
│   └── finales.py
├── verdicts/
│   ├── __init__.py
│   ├── classifieur.py     # Sigmoid winChance + verdict
│   └── assembler.py       # Construit le MoveVerdict complet
├── explanations/
│   ├── __init__.py
│   ├── templates_fr.py   # Templates en français
│   ├── templates_en.py   # Templates en anglais
│   ├── book_rag.py        # Recherche RAG dans docs/livres/
│   └── claudewriter.py    # Prompt structuré + appel Claude
├── profile/
│   ├── __init__.py
│   └── aggregator.py      # Agrège motifs sur N parties

```

```

|   └─ recommander.py      # Recommande exercices ciblés
└─ api.py                  # Endpoints FastAPI à brancher sur main.py
└─ tests/
    └─ __init__.py
    └─ fixtures/
        └─ positions.py    # Positions de référence (FEN + attendu)
        └─ dubois_*.py     # Diagrammes des livres en fixtures
        └─ games.py        # Parties complètes annotées
    └─ test_features.py
    └─ test_motif_*.py     # Un fichier par motif
    └─ test_classifier.py
    └─ test_assembler.py
    └─ test_profile.py
    └─ test_api.py

```

Intégration avec le projet existant

- **Pas de modification** des fichiers existants en dehors de `main.py` (ajout des routes) et `db/schema.py` (ajout de tables).
- Le module **importe** depuis `game_engine.py` (`GameState`, `Move`, `legal_moves`, `apply_move`) et `scan_engine.py` (`evaluate_pos`, `get_pv`).
- Le module **n'importe rien** depuis `frontend/`.
- Toute lecture de Scan se fait via `scan_engine._eval_engine` (instance sans livre interne, déjà en place).

3. Dataclasses partagées (`types.py`)

```

from dataclasses import dataclass, field
from enum import Enum
from typing import Optional

class Verdict(str, Enum):
    BRILLIANT = "brilliant"      # Sacrifice correct ET non-évident
    BEST = "best"               # Meilleur coup
    EXCELLENT = "excellent"     # Quasi-meilleur (delta < 0.03)
    GOOD = "good"               # Bon (delta < 0.075)
    INACCURACY = "inaccuracy"   # Imprécision (0.075 ≤ delta < 0.15)
    MISTAKE = "mistake"         # Erreur (0.15 ≤ delta < 0.30)
    BLUNDER = "blunder"         # Gaffe (delta ≥ 0.30)
    FORCED = "forced"           # Coup forcé (capture unique)
    BOOK = "book"               # Coup d'ouverture théorique

```

```

class Phase(str, Enum):
    OPENING = "opening"                # ≤ 12 demi-coups
    MIDDLEGAME = "middlegame"
    ENDGAME = "endgame"                # ≤ 8 pièces totales OU ≥ 1 dame de chaque côté

@dataclass
class Features:
    """Snapshot purement géométrique d'une position. Sans appel à Scan."""
    # Matériel
    white_men: int
    white_kings: int
    black_men: int
    black_kings: int
    material_balance: int                # blancs - noirs, dame = 3 pions

    # Centre / ailes
    center_count_white: int
    center_count_black: int
    left_wing_white: int
    right_wing_white: int
    left_wing_black: int
    right_wing_black: int

    # Structure
    isolated_pawns_white: list[int]
    isolated_pawns_black: list[int]
    backward_pawns_white: list[int]
    backward_pawns_black: list[int]
    holes_white: list[int]
    holes_black: list[int]
    outposts_white: list[int]
    outposts_black: list[int]

    # Mobilité
    white_legal_moves: int
    black_legal_moves: int

    # Promotion
    white_promotion_distance: int        # min(distance à rangée 1)
    black_promotion_distance: int

    # Formations détectées
    formations: list[str] = field(default_factory=list)

    # Phase
    phase: Phase = Phase.MIDDLEGAME

```

```

@dataclass
class MotifMatch:
    """Un motif détecté dans la position ou le coup joué."""
    motif: str                # ex: "coup_royal"
    role: str                 # "played" | "missed" | "suffered" | "threate
    squares: list[int]        # Cases impliquées (pour highlighting UI)
    pv: list[str]             # Variante principale qui exécute le motif
    severity: float           # 0..1 – sévérité pédagogique
    metadata: dict = field(default_factory=dict) # Données propres au motif

```

```

@dataclass
class MoveVerdict:
    """Verdict complet d'un demi-coup."""
    move_number: int
    side: str                # "white" | "black"
    move_notation: str       # ex: "32-28"
    fen_before: str
    fen_after: str
    score_before: float      # Scan, unités-pion
    score_after: float
    delta_winchance: float
    verdict: Verdict
    is_forced: bool
    motifs: list[MotifMatch] = field(default_factory=list)
    features_before: Optional[Features] = None
    features_after: Optional[Features] = None
    phase: Phase = Phase.MIDDLEGAME

```

```

@dataclass
class GameAnalysis:
    """Analyse pédagogique complète d'une partie."""
    game_id: int
    user_id: Optional[int]
    user_side: Optional[str]
    opening_name: Optional[str]
    verdicts: list[MoveVerdict]
    summary: dict            # cf. §7

```

```

@dataclass
class UserProfile:
    """Profil agrégé d'un joueur."""
    user_id: int
    games_count: int
    average_accuracy: float
    strengths: list[dict]    # [{"motif": ..., "frequency": ..., "score":
    weaknesses: list[dict]

```

```
weakest_phase: Phase
recommended_exercise_tags: list[str]
```

4. Couche 1 — Features (`pedagogy/features/`)

Principe

Fonctions **pures** prenant un `GameState` (du module `game_engine`) et retournant des scalaires, sans aucun appel à `Scan`. Testables en isolation totale.

`material.py`

```
def count_material(state) -> dict:
    """Retourne {white_men, white_kings, black_men, black_kings, balance}.
    balance = (white_men + 3*white_kings) - (black_men + 3*black_kings)."""
```

`geometry.py`

Constantes :

```
CENTER_CORE = frozenset({22, 23, 27, 28})
CENTER_EXTENDED = frozenset({16, 17, 18, 19, 21, 22, 23, 24, 26, 27, 28, 29, 31,
LEFT_WING = frozenset({6, 11, 16, 21, 26, 31, 36, 41, 46})
RIGHT_WING = frozenset({5, 10, 15, 20, 25, 30, 35, 40, 45, 50})
WHITE_PROMOTION_ROW = frozenset({1, 2, 3, 4, 5})
BLACK_PROMOTION_ROW = frozenset({46, 47, 48, 49, 50})
MAIN_DIAGONAL = (5, 10, 14, 19, 23, 28, 32, 37, 41, 46)
```

Fonctions :

```
def square_to_coords(sq: int) -> tuple[int, int]:
    """Retourne (row, col) avec row ∈ [1,10], col ∈ [1,10]. Cases sombres uniquement

def diagonal_neighbors(sq: int) -> list[int]:
    """Voisins en diagonale (0 à 4 cases)."""

def squares_between(sq_a: int, sq_b: int) -> list[int]:
    """Cases entre deux cases sur une diagonale. Lève ValueError si pas sur diago
```

`structure.py`


```

def find_isolated_pawns(state, side: str) -> list[int]:
    """Pions sans pion propre sur les 4 diagonales voisines."""

def find_backward_pawns(state, side: str) -> list[int]:
    """Pions sans pion propre devant eux (vers la direction de promotion)."""

def find_holes(state, side: str) -> list[int]:
    """Cases vides à l'intérieur de la formation, atteignables par l'adversaire e

def find_outposts(state, side: str) -> list[int]:
    """Pions avancés (rangée > 5 pour blancs, < 6 pour noirs) qui ne peuvent être

```

mobility.py

```

def count_legal_moves(state, side: str) -> int:
    """Délègue à game_engine.legal_moves(state.with_turn(side))."""

def threatened_captures(state, by_side: str) -> list[Move]:
    """Toutes les captures que `by_side` pourrait jouer si c'était son tour."""

def hanging_pieces(state, side: str) -> list[int]:
    """Pions/dames de `side` capturables par l'adversaire au prochain coup
    sans contre-capture suffisante."""

```

formations.py

```

KNOWN_FORMATIONS = {
    "classique_blancs": frozenset({32, 37, 41}),
    "classique_noirs": frozenset({10, 14, 19}),
    "roozenburg_blancs": frozenset({28, 32, 37}),
    "roozenburg_noirs": frozenset({14, 19, 23}),
    "ghestem_blancs": frozenset({27, 32, 38}),
    # À compléter au fur et à mesure
}

def detect_formations(state) -> list[str]:
    """Retourne les noms de formations présentes (sous-ensemble de KNOWN_FORMATIO

```

Fonction agrégatrice

```

def compute_features(state) -> Features:
    """Compose toutes les fonctions ci-dessus. Pas d'appel Scan."""

```

Détermination de la phase

```
def determine_phase(state, half_move: int) -> Phase:
    total_pieces = sum(count_material(state).values()) # men + kings
    has_kings_both_sides = (state.white_kings >= 1 and state.black_kings >= 1)
    if half_move <= 12:
        return Phase.OPENING
    if total_pieces <= 8 or has_kings_both_sides:
        return Phase.ENDGAME
    return Phase.MIDDLEGAME
```

Tests features

Au moins 25 tests couvrant :

- Position initiale → toutes les valeurs attendues.
- 5 positions de référence du livre Dubois (à recopier en FEN dans `fixtures/positions.py`).
- Cas limites : damier vide sauf 2 pièces, formation classique pure, position en finale dame contre 3 pions.

5. Couche 2 — Motifs (`pedagogy/motifs/`)

Classe de base

```
# motifs/base.py
from abc import ABC, abstractmethod

class MotifDetector(ABC):
    """Chaque détecteur de motif hérite de cette classe."""
    name: str # ex: "coup_royal"
    requires_pv: bool = True # True si on a besoin de la variante principale

    @abstractmethod
    def detect(self,
               state_before,
               move,
               state_after,
               pv: list[str] | None = None,
               scan_score_before: float = 0.0,
               scan_score_after: float = 0.0) -> MotifMatch | None:
```

```

        """Retourne MotifMatch si détecté, None sinon."""

    @abstractmethod
    def detect_missed(self,
                      state_before,
                      best_move,
                      best_pv: list[str],
                      played_move) -> MotifMatch | None:
        """Détecte le motif dans le meilleur coup que le joueur N'A PAS joué."""

```

coup_royal.py — algorithme détaillé

Signature : une rafle qui capture ≥ 6 pions, dont la majorité au centre, sur un axe diagonal long passant par la grande diagonale.

```

class CoupRoyalDetector(MotifDetector):
    name = "coup_royal"

    def _is_royal_signature(self, captures: list[int]) -> bool:
        if len(captures) < 6:
            return False
        center_captures = sum(1 for sq in captures if sq in CENTER_EXTENDED)
        if center_captures < 4:
            return False
        # La rafle doit s'étendre sur  $\geq 4$  rangées différentes
        rows = {square_to_coords(sq)[0] for sq in captures}
        if len(rows) < 4:
            return False
        # La rafle finit dans la rangée de promotion OU traverse la grande diagon
        return True # cf. règles fines ci-dessous

    def detect(self, state_before, move, state_after, pv=None, **kw):
        if not move.captures or len(move.captures) < 6:
            return None
        if not self._is_royal_signature(move.captures):
            return None
        return MotifMatch(
            motif="coup_royal",
            role="played",
            squares=move.path + move.captures,
            pv=[notation(move)],
            severity=min(1.0, len(move.captures) / 10.0),
            metadata={"captures_count": len(move.captures)}
        )

    def detect_missed(self, state_before, best_move, best_pv, played_move):

```

```

# Le coup royal était disponible mais le joueur a fait autre chose
if not best_move.captures or len(best_move.captures) < 6:
    return None
if played_move.path == best_move.path:
    return None # Il l'a joué
if not self._is_royal_signature(best_move.captures):
    return None
return MotifMatch(
    motif="coup_royal",
    role="missed",
    squares=best_move.path + best_move.captures,
    pv=[notation(best_move)],
    severity=1.0,
    metadata={"captures_count": len(best_move.captures)}
)

```

coup_turc.py

Signature : dans la séquence de rafle, une pièce adverse est *survolée* (donc capturée logiquement) deux fois.

Note d’implémentation : en règles FMJD, on ne peut pas capturer la même pièce deux fois — mais le “coup turc” désigne historiquement les rafles dont la **trajectoire** revient sur ses pas, créant l’illusion d’une double prise. L’implémentation doit donc détecter : **trajectoire qui passe deux fois par la même case intermédiaire**.

```

def _is_turc_signature(path: list[int]) -> bool:
    """Le chemin (cases successives) repasse par une même case."""
    return len(path) != len(set(path))

```

coup_de_talon.py

Signature : rafle qui inclut un retour en arrière (renversement de direction sur la diagonale).

```

def _has_heel_motion(path: list[int]) -> bool:
    """Au moins un renversement de direction sur la diagonale dans le chemin."""
    if len(path) < 3:
        return False
    directions = []
    for a, b in zip(path, path[1:]):
        ra, ca = square_to_coords(a)
        rb, cb = square_to_coords(b)
        directions.append((rb - ra > 0, cb - ca > 0)) # (down?, right?)
    # Renversement = deux directions consécutives opposées sur un axe
    for d1, d2 in zip(directions, directions[1:]):

```

```

        if d1 != d2:
            return True
    return False

```

envoi_a_dame.py

Signature : sacrifice (delta matériel ≤ -1) qui amène une promotion forcée dans les 2 demi-coups suivants ET un score Scan positif après.

Implémentation : examiner la PV de Scan, vérifier qu'elle contient un coup vers une case de promotion + que le pion qui promeut existe bien et est forcé par les règles de prise.

sacrifices.py

```

class SacrificeDetector(MotifDetector):
    name = "sacrifice"

    def detect(self, state_before, move, state_after, pv, scan_score_before, scan_score_after):
        bal_before = compute_features(state_before).material_balance
        bal_after = compute_features(state_after).material_balance
        side_sign = 1 if state_before.turn == "white" else -1
        material_loss = side_sign * (bal_after - bal_before)
        if material_loss >= 0:
            return None # Pas de sacrifice
        # Le score Scan reste favorable ou neutre côté joueur
        score_for_player = side_sign * scan_score_after
        if score_for_player < -0.3:
            return None # Mauvais sacrifice
        return MotifMatch(
            motif="sacrifice",
            role="played",
            squares=move.path,
            pv=pv or [],
            severity=abs(material_loss) / 3.0,
            metadata={
                "material_loss": abs(material_loss),
                "score_after": scan_score_after
            }
        )

```

prise_max_ratee.py

Cas particulier : le joueur a joué une prise, mais pas la plus longue. Le `game_engine` empêche normalement ce cas — mais en import PDN d'une partie d'un autre site avec règles différentes, ça peut arriver. Détecter et signaler.

```

class PriseMaxRateeDetector(MotifDetector):
    name = "prise_max_ratee"
    requires_pv = False

    def detect(self, state_before, move, state_after, **kw):
        if not move.captures:
            return None
        all_legal = legal_moves(state_before)
        max_captures = max((len(m.captures) for m in all_legal), default=0)
        if len(move.captures) >= max_captures:
            return None
        return MotifMatch(
            motif="prise_max_ratee",
            role="played",
            squares=move.path,
            pv=[],
            severity=1.0,
            metadata={
                "captures_played": len(move.captures),
                "captures_possible": max_captures
            }
        )

```

enchainements.py

Détecte les pions adverses *cloués* entre deux pions du joueur. Concept stratégique, pas tactique.

percee.py

Pion à 2 coups de la promotion sans interception possible. Calcul : tracer le chemin du pion vers sa rangée de promotion, vérifier qu'aucune pièce adverse ne peut s'interposer en ≤ 2 coups.

finales.py

Plusieurs sous-détecteurs :

- `opposition_dame_pion` : finale dame contre pion, position d'opposition
- `dame_contre_3_pions_diagonale` : finale gagnante connue
- `coup_de_l_escalier` : technique de fin de partie

Registre

```

# motifs/__init__.py
from .coup_royal import CoupRoyalDetector
from .coup_turc import CoupTurcDetector
# ...

ALL_DETECTORS: list[type[MotifDetector]] = [
    CoupRoyalDetector,
    CoupTurcDetector,
    CoupDeTalonDetector,
    EnvoiADameDetector,
    SacrificeDetector,
    PriseMaxRateeDetector,
    EnchainementDetector,
    PerceeDetector,
    # ...
]

def detect_all(state_before, move, state_after, pv, scores) -> list[MotifMatch]:
    matches = []
    for cls in ALL_DETECTORS:
        det = cls()
        m = det.detect(state_before, move, state_after, pv, *scores)
        if m:
            matches.append(m)
    return matches

```

Tests motifs

Critère d'acceptation par motif : minimum 10 tests, dont au moins 5 positifs (motif présent) et 5 négatifs (motif absent), tirés des livres du dossier `docs/livres/` . Pour le coup royal, viser 20+ tests, c'est le motif le plus complexe.

Format de fixture :

```

# tests/fixtures/dubois_coup_royal.py
COUP_ROYAL_FIXTURES = [
    {
        "name": "Dubois D5 - coup royal classique",
        "fen_before": "W:W31,32,33,38,39,40,41,42,46,47:B7,8,9,12,13,14,16,17,18,
        "move": [(40, 34), (34, 24), (24, 14), (14, 3)],
        "expected_detected": True,
        "expected_captures": 6,
        "expected_role": "played",
    },
    # ...
]

```

6. Couche 3 — Verdicts (`pedagogy/verdicts/`)

`classifier.py`

Reprend la sigmoïde **strictement identique** à `frontend/src/lib/gameAnnotations.ts` pour cohérence d’affichage :

```
import math

def win_chance(score: float) -> float:
    """Probabilité de victoire normalisée [-1, +1]. score en unités-pion Scan."""
    return 2.0 / (1.0 + math.exp(-2.0 * score)) - 1.0

def classify_move(score_before: float,
                 score_after: float,
                 side: str,
                 is_forced: bool,
                 is_book: bool,
                 motifs: list[MotifMatch]) -> Verdict:
    if is_book:
        return Verdict.BOOK
    if is_forced:
        return Verdict.FORCED
    sign = 1 if side == "white" else -1
    wc_before = sign * win_chance(score_before)
    wc_after = sign * win_chance(score_after)
    delta = wc_before - wc_after # Positif si la position s'est dégradée

    # Brillant : sacrifice détecté ET pas de perte de winchance
    if any(m.motif == "sacrifice" and m.role == "played" for m in motifs):
        if delta < 0.05:
            return Verdict.BRILLIANT

    if delta < 0.01:
        return Verdict.BEST
    if delta < 0.03:
        return Verdict.EXCELLENT
    if delta < 0.075:
        return Verdict.GOOD
    if delta < 0.15:
        return Verdict.INACCURACY
    if delta < 0.30:
        return Verdict.MISTAKE
```



```
return Verdict.BLUNDER
```

assembler.py

```
def assemble_verdict(state_before,
                     move,
                     state_after,
                     score_before: float,
                     score_after: float,
                     best_move,
                     best_pv: list[str],
                     half_move_number: int,
                     is_book: bool) -> MoveVerdict:
    """Assemble le MoveVerdict complet d'un demi-coup."""
    is_forced = len(legal_moves(state_before)) == 1
    motifs = detect_all(state_before, move, state_after,
                        best_pv, (score_before, score_after))

    # Détection des motifs manqués
    if move.path != best_move.path:
        for cls in ALL_DETECTORS:
            det = cls()
            missed = det.detect_missed(state_before, best_move, best_pv, move)
            if missed:
                motifs.append(missed)

    verdict = classify_move(score_before, score_after,
                           state_before.turn, is_forced, is_book, motifs)
    phase = determine_phase(state_before, half_move_number)

    return MoveVerdict(
        move_number=half_move_number,
        side=state_before.turn,
        move_notation=notation(move),
        fen_before=state_to_fen(state_before),
        fen_after=state_to_fen(state_after),
        score_before=score_before,
        score_after=score_after,
        delta_winchance=...,
        verdict=verdict,
        is_forced=is_forced,
        motifs=motifs,
        features_before=compute_features(state_before),
        features_after=compute_features(state_after),
        phase=phase,
    )
```

7. Couche 4 — Explications (`pedagogy/explanations/`)

Stratégie en 3 couches empilables

L'API expose un paramètre `explanation_mode` ∈ {`"template"`, `"template+book"`, `"claude"`} . L'utilisateur (ou l'admin) choisit le niveau.

`templates_fr.py`

```
TEMPLATES_FR: dict[tuple[str, str, Verdict | None], str] = {
    # (motif, role, verdict ou None) -> template

    ("coup_royal", "played", Verdict.BRILLIANT):
        "Magnifique coup royal ! Vous avez exécuté {captures_count} prises en une "
        "un mécanisme classique nommé d'après sa puissance dévastatrice au centre

    ("coup_royal", "missed", Verdict.BLUNDER):
        "Vous avez raté un coup royal ! La séquence {pv} aurait capturé {captures "
        "et probablement gagné la partie. Le coup royal est un schéma à toujours "
        "quand votre adversaire empile des pions au centre.",

    ("coup_royal", "suffered", Verdict.BLUNDER):
        "Coup royal subi : votre adversaire a pu exécuter une rafle de {captures_ "
        "À l'avenir, méfiez-vous des configurations centrales avec des pions sur

    ("sacrifice", "played", Verdict.BRILLIANT):
        "Beau sacrifice ! Vous offrez {material_loss} pion(s), mais Scan évalue t "
        "votre position à {score_after:+.1f}. Bien vu.",

    ("prise_max_ratee", "played", None):
        "Attention : vous avez joué une prise de {captures_played} pions, "
        "mais la règle de prise maximale obligeait à en prendre {captures_possibl "
        "En FMJD, la séquence la plus longue est imposée.",

    # ... un template par couple (motif, role, verdict) significatif
}

def render_template(motif: MotifMatch, verdict: Verdict, ctx: dict) -> str | None
    key_specific = (motif.motif, motif.role, verdict)
    key_generic = (motif.motif, motif.role, None)
    template = TEMPLATES_FR.get(key_specific) or TEMPLATES_FR.get(key_generic)
    if template is None:
```

```
        return None
    return template.format(**motif.metadata, **ctx)
```

book_rag.py

```
class BookRAG:
    """Indexe les PDF de docs/livres/ et retrouve un passage par tag de motif."""

    def __init__(self, books_dir: Path):
        self.books_dir = books_dir
        self._index = None  # Lazy load

    def search(self, motif_name: str, max_results: int = 1) -> list[dict]:
        """Retourne [{book, page, excerpt, score}, ...]."""
```

Implémentation v1 minimale : indexation par mots-clés (TF-IDF avec `scikit-learn`) sur les PDF convertis en texte au démarrage. Pas d'embeddings en v1 — c'est suffisant pour des motifs nommés rares dans le texte.

Implémentation v2 : embeddings via `sentence-transformers` (modèle `paraphrase-multilingual-MiniLM-L12-v2`), FAISS pour la recherche. Optionnel, derrière feature flag.

claude_writer.py — Le point critique

Règle d'or : Claude reçoit une liste *fermée* de motifs et a interdiction d'en ajouter.

```
SYSTEM_PROMPT = """Tu es un commentateur pédagogique pour le jeu de dames interna
```

Tu reçois pour un coup donné :

1. La notation du coup joué (ex: "32-28")
2. Le verdict ("blunder", "mistake", "brilliant", etc.)
3. Une LISTE FERMÉE de motifs détectés par un système déterministe
4. Des features positionnelles (centre, formations, etc.)
5. Optionnellement, des extraits de livres pédagogiques

Tu dois rédiger un commentaire en français de 1 à 3 phrases, au ton bienveillant

CONTRAINTES STRICTES :

- Tu ne dois mentionner AUCUN motif tactique qui n'est pas dans la liste fournie.
- Tu ne dois pas inventer de variante (séquence de coups). Si tu cites une variante tu ne peux le faire qu'en recopiant exactement la PV fournie.
- Tu ne dois pas évaluer la position avec un score différent de celui fourni par
- Tu ne dois pas faire de prédiction sur ce que jouera l'adversaire après.
- Si la liste de motifs est vide, contente-toi de commenter le verdict en termes généraux (centre, mobilité) sans nommer de motif tactique.

Tu réponds UNIQUEMENT par le commentaire, sans préambule ni signature."""

```
def build_user_prompt(verdict: MoveVerdict, book_excerpts: list[str] = None) -> str:
    motifs_block = "\n".join(
        f"- {m.motif} ({m.role}) - sévérité {m.severity:.2f} - métadonnées {m.metadata}"
        for m in verdict.motifs
    ) or "(aucun motif détecté)"

    features_block = (
        f"Centre blancs/noirs: {verdict.features_before.center_count_white}"
        f"/{verdict.features_before.center_count_black}, "
        f"Matériel: {verdict.features_before.material_balance:+d}, "
        f"Formations: {verdict.features_before.formations or 'aucune'}, "
        f"Phase: {verdict.phase}"
    )

    excerpts_block = "\n".join(f"> {e}" for e in (book_excerpts or [])) or "(aucun extrait)"

    return f"""COUP JOUÉ: {verdict.move_notation} ({verdict.side})
VERDICT: {verdict.verdict}
SCORES SCAN: {verdict.score_before:+.2f} -> {verdict.score_after:+.2f}

MOTIFS DÉTECTÉS:
{motifs_block}

POSITION:
{features_block}

EXTRAITS PÉDAGOGIQUES PERTINENTS:
{excerpts_block}

Rédige le commentaire."""
```

```
async def write_commentary(verdict: MoveVerdict, book_rag: BookRAG | None) -> str:
    excerpts = []
    if book_rag:
        for motif in verdict.motifs:
            results = book_rag.search(motif.motif, max_results=1)
            excerpts.extend(r["excerpt"] for r in results)

    client = AsyncAnthropic()
    response = await client.messages.create(
        model="claude-opus-4-7",
        max_tokens=300,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": build_user_prompt(verdict, excerpts)}
```

```

)
text = response.content[0].text.strip()
return _verify_no_invented_motifs(text, verdict)

def _verify_no_invented_motifs(text: str, verdict: MoveVerdict) -> str:
    """Vérifie que le texte ne mentionne pas de motif non détecté.
    Retourne le texte tel quel si OK, sinon le tronque à la première phrase neutr
    KNOWN_MOTIFS_FR = {
        "coup royal", "coup turc", "coup de talon", "envoi à dame",
        "coup philippe", "coup raphaël", "coup de l'express", "coup bonnard",
        "sacrifice", "enchaînement", "percée", "opposition",
    }
    detected_names = {m.motif.replace("_", " ") for m in verdict.motifs}
    text_lower = text.lower()
    for motif_name in KNOWN_MOTIFS_FR:
        if motif_name in text_lower and motif_name.replace(" ", "_") not in {m.mo
            # Hallucination détectée
            return _fallback_template(verdict)
    return text

```

Pipeline complet

```

async def explain_verdict(verdict: MoveVerdict,
                          mode: str = "template",
                          book_rag: BookRAG | None = None,
                          lang: str = "fr") -> str:
    if mode == "template":
        return _render_from_templates(verdict, lang)
    if mode == "template+book":
        base = _render_from_templates(verdict, lang)
        excerpts = []
        for m in verdict.motifs:
            excerpts.extend(book_rag.search(m.motif, max_results=1))
        if excerpts:
            base += f"\n\nPour approfondir : {excerpts[0]['book']} p.{excerpts[0]
        return base
    if mode == "claude":
        return await write_commentary(verdict, book_rag)
    raise ValueError(f"Unknown mode: {mode}")

```

8. Couche 5 — Profil utilisateur (`pedagogy/profile/`)


```

return UserProfile(
    user_id=user_id,
    games_count=len(games),
    average_accuracy=sum(accuracy_scores) / max(len(accuracy_scores), 1),
    strengths=strengths,
    weaknesses=weaknesses,
    weakest_phase=weakest_phase,
    recommended_exercise_tags=[w["motif"] for w in weaknesses],
)

```

recommender.py

```

def recommend_exercises(profile: UserProfile,
                        n: int = 10,
                        db_conn = None) -> list[dict]:
    """Sélectionne n exercices ciblés sur les faiblesses du joueur."""
    weak_tags = profile.recommended_exercise_tags
    excluded = fetch_already_solved_exercise_ids(profile.user_id, db_conn)
    exercises = fetch_exercises_by_tags(weak_tags, exclude_ids=excluded, db_conn=
    return exercises[:n]

```

Cela nécessite que les exercices existants reçoivent des **tags** (champ déjà présent dans la table `exercises` ? À vérifier ; sinon, ajouter via migration).

9. Endpoints API (`pedagogy/api.py`)

À brancher dans `main.py` via `app.include_router(pedagogy_router)`.

```

from fastapi import APIRouter, Depends, HTTPException

router = APIRouter(prefix="/api/pedagogy", tags=["pedagogy"])

@router.post("/analyze-game")
async def analyze_game(req: AnalyzeGameRequest, user = Depends(current_user_optio
    """Analyse pédagogique complète d'une partie (PDN ou game_id).
    Réutilise opening_book_db pour le cache."""
    # Rate-limit 3/min (cf. main.py)

@router.get("/move-verdict/{game_id}/{move_number}")
async def get_move_verdict(game_id: int, move_number: int):
    """Retourne le MoveVerdict d'un demi-coup spécifique."""

@router.post("/explain-move")

```

```

async def explain_move(req: ExplainMoveRequest):
    """Génère le commentaire d'un coup, en mode template / template+book / claude
    # Rate-limit 5/min en mode claude, 60/min en mode template

@router.get("/profile/{user_id}")
async def get_user_profile(user_id: int, user = Depends(current_user)):
    """Profil agrégé d'un utilisateur. Auth obligatoire, user_id doit == user.id.
    if user.id != user_id and not user.is_admin:
        raise HTTPException(403)

@router.get("/profile/me/recommendations")
async def get_recommendations(n: int = 10, user = Depends(current_user)):
    """Exercices recommandés à l'utilisateur courant."""

```

Modèles Pydantic à ajouter dans `models.py` (ou un `pedagogy/models.py` séparé).

Réponses sérialisées depuis les dataclasses via `dataclasses.asdict`.

10. Schéma de base de données

Migrations à ajouter à `db/schema.py`

```

CREATE TABLE IF NOT EXISTS move_verdicts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    game_id INTEGER NOT NULL,
    move_number INTEGER NOT NULL,
    side TEXT NOT NULL,
    fen_before TEXT NOT NULL,
    fen_after TEXT NOT NULL,
    move_notation TEXT NOT NULL,
    score_before REAL NOT NULL,
    score_after REAL NOT NULL,
    delta_winchance REAL NOT NULL,
    verdict TEXT NOT NULL,
    is_forced INTEGER NOT NULL,
    phase TEXT NOT NULL,
    motifs_json TEXT NOT NULL,
    features_json TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    UNIQUE (game_id, move_number)
);

CREATE INDEX IF NOT EXISTS idx_move_verdicts_game ON move_verdicts(game_id);

```



```

CREATE INDEX IF NOT EXISTS idx_move_verdicts_verdict ON move_verdicts(verdict);

-- Index sur motifs via JSON_EXTRACT pour les requêtes profil
CREATE INDEX IF NOT EXISTS idx_move_verdicts_motifs
    ON move_verdicts(json_extract(motifs_json, '$'));

CREATE TABLE IF NOT EXISTS pedagogy_explanations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    move_verdict_id INTEGER NOT NULL,
    mode TEXT NOT NULL,          -- "template" | "template+book" | "claudio"
    lang TEXT NOT NULL,
    text TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (move_verdict_id) REFERENCES move_verdicts(id) ON DELETE CASCADE,
    UNIQUE (move_verdict_id, mode, lang)
);

-- Tags d'exercices, pour le recommander
CREATE TABLE IF NOT EXISTS exercise_tags (
    exercise_id INTEGER NOT NULL,
    tag TEXT NOT NULL,
    PRIMARY KEY (exercise_id, tag),
    FOREIGN KEY (exercise_id) REFERENCES exercises(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_exercise_tags_tag ON exercise_tags(tag);

```

Migration des exercices existants

Script `pedagogy/scripts/tag_existing_exercises.py` : parcourt tous les exercices, applique les détecteurs sur la position de départ + la solution, écrit les tags détectés dans `exercise_tags`. Idempotent.

11. Stratégie de test

Pyramide

- **Tests unitaires** (95% des tests) : chaque détecteur de feature et de motif, isolé, avec fixtures.
- **Tests d'intégration** : `assemble_verdict` sur des positions complètes, en mockant Scan.
- **Tests end-to-end** : 5 parties complètes annotées à la main par un joueur (à fournir via

fixtures), vérifier que les verdicts assemblés correspondent.

Fixtures à construire

1. `fixtures/positions.py` : 30 positions de référence en FEN (Dubois, parties classiques).
2. `fixtures/dubois_coup_royal.py` : 20 diagrammes du livre Dubois sur le coup royal.
3. `fixtures/dubois_*.py` : un fichier par chapitre pertinent.
4. `fixtures/games.py` : 5 parties complètes en PDN avec verdicts attendus pour chaque coup.

Mock de Scan

`tests/conftest.py` fournit un `MockScanEngine` qui retourne des scores et PV pré-calculés. Aucun test pédagogique ne doit lancer le processus Scan réel.

Critères de couverture

- `pytest --cov=pedagogy` : couverture $\geq 85\%$.
 - Tous les motifs P1 doivent avoir 100% de couverture de leurs méthodes `detect` et `detect_missed`.
-

12. Plan de livraison incrémental

L'agent doit livrer **dans cet ordre**, avec PR séparées et tests verts à chaque étape.

PR 1 — Squelette + types + features de base

- `pedagogy/__init__.py`, `types.py`
- `features/material.py`, `features/geometry.py`, `features/mobility.py`
- `tests/test_features.py` (15+ tests)
- **Critère** : `compute_features(initial_state)` retourne les valeurs attendues.

PR 2 — Features avancées

- `features/structure.py`, `features/formations.py`
- 10+ tests supplémentaires

- **Critère** : détection correcte sur 5 positions Dubois.

PR 3 — Motifs P1 partie 1

- `motifs/base.py`, `motifs/coup_royal.py`, `motifs/prise_max_ratee.py`
- Fixtures Dubois pour ces motifs
- **Critère** : 20 tests verts sur le coup royal.

PR 4 — Motifs P1 partie 2

- `motifs/coup_turc.py`, `motifs/coup_de_talon.py`, `motifs/envoi_a_dame.py`,
`motifs/sacrifices.py`
- **Critère** : 40+ tests motifs verts au total.

PR 5 — Classifier + Assembler

- `verdicts/classifier.py`, `verdicts/assembler.py`
- Mock Scan dans `conftest`
- **Critère** : assemblage correct sur une partie de référence complète.

PR 6 — Templates FR

- `explanations/templates_fr.py`
- Tests : tous les couples (motif, role, verdict) ont un template ou tombent dans un fallback documenté.

PR 7 — Schéma DB + persistance

- Migrations dans `db/schema.py`
- Fonctions de lecture/écriture dans `pedagogy/storage.py`
- Tests de round-trip.

PR 8 — Endpoints API

- `pedagogy/api.py` + branchement dans `main.py`
- Tests API avec `httpx.AsyncClient`
- **Critère** : `/api/pedagogy/analyze-game` fonctionne bout-en-bout en mock Scan.

PR 9 — Profil utilisateur

- `profile/aggregator.py`, `profile/recommender.py`
- Tests sur jeu de 10 parties simulées.

PR 10 — Claude writer + vérification anti-hallucination

- `explanations/claude_writer.py`
- Tests : 20 prompts, vérifier qu'aucun motif inventé ne passe.

PR 11 — BookRAG (TF-IDF v1)

- `explanations/book_rag.py`
- Indexation au démarrage du backend.

PR 12 — Templates EN + i18n

- `explanations/templates_en.py`
- Paramètre `lang` propagé partout.

PR 13 — Script de tagging des exercices existants

- `pedagogy/scripts/tag_existing_exercises.py`
- À lancer une fois en prod.

PR 14 — Motifs P2

- Coup Philippe, Raphaël, l'Express, Bonnard.

13. Conventions de code

- **Python 3.11+** : utiliser `match`, `|` pour types optionnels, `dataclasses`.
- **Type hints partout**. Vérifier avec `mypy --strict pedagogy/`.
- **Docstrings** style Google sur toute fonction publique.
- **Pas de print** : utiliser `logging` (`logger pedagogy.<module>`).
- **Pas de variables globales mutables** dans `pedagogy/`. Tous les détecteurs sont `stateless`.

- **Imports** : isort + black, ligne max 100 caractères.
 - **Tests** : un fichier par module testé, nom miroir (`test_x.py` pour `x.py`).
 - **Commits** : conventional commits (`feat:` , `fix:` , `test:` , `refactor:`).
-

14. Points d'attention spécifiques

14.1 Cohérence FEN

Le projet utilise déjà une convention FEN précise (cf. `scan_engine.py`). Vérifier impérativement que `state_to_fen` produit la même chaîne que celle envoyée à Scan, sinon les caches `opening_book_db` et `move_verdicts` divergent.

14.2 Symétrie horizontale

Le module `opening_book_db.py` canonicalise les positions par symétrie. Le module `pedagogy` ne doit **pas** canonicaliser : on veut le contexte exact du joueur (un coup sur l'aile gauche n'est pas pédagogiquement identique à son symétrique sur l'aile droite, même s'ils ont la même évaluation).

14.3 Coup de la PV multi-coups

Quand Scan renvoie une PV de plusieurs coups, les détecteurs `detect_missed` doivent vérifier que **le motif s'exécute dans le premier coup de la PV** (pas plus loin), sinon le commentaire devient hors-sujet pour le joueur.

14.4 Performance

`compute_features` est appelé deux fois par demi-coup. Pour une partie de 80 demi-coups, c'est 160 appels. Cible : < 1 ms par appel. Profiler avec `cProfile` à la fin de la PR 2.

14.5 Compatibilité avec le frontend existant

`AnalysisPanel.tsx` et `gameAnnotations.ts` attendent un format précis. Ne pas casser la rétrocompatibilité : les nouveaux endpoints sont **additifs**, le frontend existant continue de marcher.

14.6 Rate limiting

Le mode `claude` consomme l'API Anthropic. Réutiliser le rate limiter en mémoire existant (5/min) sur l'endpoint `/explain-move` avec `mode=claude` . En mode `template` , ne pas rate-limiter (60/min de courtoisie suffit).

14.7 Hallucinations LLM — politique zéro tolérance

Le test `test_no_invented_motifs` doit être bloquant. Soumettre à Claude des verdicts avec liste vide de motifs, vérifier que la réponse ne contient *aucun* mot-clé tactique reconnu. Si ça arrive : améliorer le prompt ou tomber en fallback template.

15. Définition de “fini”

Le framework est considéré comme livré quand :

- Les 14 PR ci-dessus sont mergées.
 - `pytest backend/pedagogy/` : 100% verts, couverture $\geq 85\%$.
 - `mypy --strict backend/pedagogy/` : 0 erreur.
 - Endpoint `/api/pedagogy/analyze-game` traite une partie de 80 demi-coups en < 30 s (hors temps Scan).
 - Sur 20 parties tests issues de Lidraughts, au moins 80% des coups taggés `BLUNDER` reçoivent au moins un motif détecté.
 - Un README `backend/pedagogy/README.md` documente l'architecture, comment ajouter un motif, comment ajouter un template.
 - Le frontend existant continue de fonctionner sans modification.
-

16. Ressources fournies

L'agent peut consulter :

- `docs/livres/dubois_perfectionnement_combinaisons_V4.pdf` (motifs, théorie, diagrammes annotés)
- `docs/livres/dubois_combinaisons.pdf`
- `docs/livres/dubois_ouvertures.pdf`
- `docs/livres/dubois_fins_de_parties.pdf`
- `docs/parties/*.pdn` (parties de référence)
- Le code existant `backend/` (en lecture seule).

L'agent **n'a pas** accès à internet. Toutes les références doivent être tirées des livres fournis.

17. Premiers pas concrets

À l'ouverture du repo, l'agent doit :

1. Lire `Read me projet` (ou `README.md`) pour le contexte.
 2. Lire `backend/game_engine.py` pour le modèle de données.
 3. Lire `backend/scan_engine.py` pour l'interface Scan.
 4. Lire `backend/db/schema.py` pour la structure SQLite.
 5. Lire `frontend/src/lib/gameAnnotations.ts` pour la cohérence verdict/sigmoïde.
 6. Créer la branche `feat/pedagogy-framework`.
 7. Commencer par la PR 1.
-

18. PR additionnelle — Prospection multi-LLM (à insérer entre PR 10 et PR 11)

Objectif

Le mode `claude` du writer (PR 10) utilise par défaut `claude-opus-4-7`, le modèle le plus cher du catalogue Anthropic. Pour une analyse complète d'une partie de 80 demi-coups, on appelle l'API 80 fois, soit potentiellement $80 \times (\text{input} \sim 800 \text{ tokens} + \text{output} \sim 200 \text{ tokens}) =$ environ 80 000 tokens. À 5 \$/M input et 25 \$/M output, ça représente environ **0.72 € par partie analysée**. Sur 1000 parties/mois ça fait 720 € de facture pour de la simple rédaction.

L'objectif de cette PR est d'introduire une **couche d'abstraction sur les LLM** permettant de tester et de router vers le modèle le moins cher qui maintient un niveau de qualité acceptable.

Modèles à tester

Sélection au prix de mai 2026 (à actualiser à l'implémentation) :

Provider	Modèle	Input \$/M	Output \$/M	Coût/partie (estim.)

Anthropic	claude-opus-4-7	5.00	25.00	référence
Anthropic	claude-sonnet-4-6	3.00	15.00	~60%
Anthropic	claude-haiku-4-5	0.25	1.25	~5%
OpenAI	gpt-5.4-mini	~0.40	~1.60	~7%
OpenAI	gpt-4.1-nano	0.10	0.40	~2%
Google	gemini-2.5-flash	0.30	2.50	~10%
Google	gemini-2.5-pro	1.25	10.00	~40%
DeepSeek	deepseek-v3.2	0.14	0.28	~1%
Mistral	mistral-small-3.2	0.10	0.30	~1.5%
Local (option)	mistral-7b-instruct (vLLM, GPU 1xA10)	—	—	infrastructure

Note : les prix changent vite (baisse de ~80% sur 12 mois en 2025-2026). L'agent doit vérifier les prix officiels au moment de l'implémentation et noter la date dans le rapport.

Architecture — abstraction LLM

```
# pedagogy/explanations/llm_client.py

from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class LLMResponse:
    text: str
    input_tokens: int
    output_tokens: int
    latency_ms: int
    model_id: str
    cost_usd: float

class LLMClient(ABC):
    model_id: str
    input_price_per_million: float
    output_price_per_million: float

    @abstractmethod
```



```

        async def generate(self, system: str, user: str, max_tokens: int = 300) -> LL
        ...

class AnthropicClient(LLMClient):
    """Wrap AsyncAnthropic - modèles claude-opus-*, claude-sonnet-*, claude-haiku

class OpenAIClient(LLMClient):
    """Wrap AsyncOpenAI - modèles gpt-*, o*."""

class GoogleClient(LLMClient):
    """Wrap google-generativeai - modèles gemini-*."""

class DeepSeekClient(LLMClient):
    """API DeepSeek (compatible OpenAI v1)."""

class MistralClient(LLMClient):
    """Wrap mistralai SDK."""

class LocalVLLMClient(LLMClient):
    """Appel HTTP vers un serveur vLLM local. input/output_price = 0."""

def get_client(model_id: str) -> LLMClient:
    """Factory. Lit la clé API depuis l'environnement selon le provider."""

```

Variables d'environnement à ajouter

```

PEDAGOGY_LLM_MODEL=claude-haiku-4-5      # Modèle par défaut pour le writer
ANTHROPIC_API_KEY=...                    # Déjà présent
OPENAI_API_KEY=...
GOOGLE_API_KEY=...
DEEPSEEK_API_KEY=...
MISTRAL_API_KEY=...
PEDAGOGY_LOCAL_VLLM_URL=http://localhost:8000/v1 # Optionnel

```

Bench : `pedagogy/scripts/bench_llm.py`

Script de benchmark autonome :

```

"""
Compare plusieurs LLM sur la tâche de rédaction pédagogique du writer.

Usage :
python -m pedagogy.scripts.bench_llm \
    --dataset tests/fixtures/bench_dataset.json \
    --models claude-opus-4-7 claude-haiku-4-5 deepseek-v3.2 gpt-4.1-nano \

```

```

--output bench_results.json
"""

import asyncio, json, time
from pathlib import Path

async def bench_one_model(client: LLMClient, dataset: list[dict]) -> dict:
    """Exécute le writer sur chaque cas du dataset, mesure coût/latence/qualité."""
    results = []
    for case in dataset:
        verdict = MoveVerdict(**case["verdict"])
        system = SYSTEM_PROMPT
        user = build_user_prompt(verdict, case.get("excerpts", []))
        t0 = time.time()
        resp = await client.generate(system, user, max_tokens=300)
        elapsed = (time.time() - t0) * 1000
        results.append({
            "case_id": case["id"],
            "text": resp.text,
            "latency_ms": elapsed,
            "cost_usd": resp.cost_usd,
            "input_tokens": resp.input_tokens,
            "output_tokens": resp.output_tokens,
            "expected_motifs": case["expected_motifs"],
            "forbidden_motifs": case["forbidden_motifs"],
        })
    return {
        "model": client.model_id,
        "total_cost_usd": sum(r["cost_usd"] for r in results),
        "p50_latency_ms": _percentile([r["latency_ms"] for r in results], 50),
        "p95_latency_ms": _percentile([r["latency_ms"] for r in results], 95),
        "cases": results,
    }

```

Dataset de benchmark

tests/fixtures/bench_dataset.json — 30 cas couvrant :

- 10 coups de différents verdicts (brilliant, blunder, forced, book...)
- 5 cas multi-motifs (coup royal + sacrifice par exemple)
- 5 cas sans aucun motif détecté (test anti-hallucination)
- 5 cas en finale (concepts stratégiques)
- 5 cas en ouverture (références théoriques)

Chaque cas spécifie :

```
{
  "id": "case_001_blunder_coup_royal_missed",
  "verdict": { ... MoveVerdict en JSON ... },
  "excerpts": ["Le coup royal est un mécanisme..."],
  "expected_motifs": ["coup_royal"],
  "forbidden_motifs": ["coup_turc", "coup_de_talon", "envoi_a_dame"],
  "expected_keywords": ["royal", "rafle", "centre"],
  "min_words": 20,
  "max_words": 80
}
```

Métriques d'évaluation

Pour chaque modèle, 4 scores sur 100 :

1. **Justesse (anti-hallucination)** — pourcentage de réponses sans aucun mot-clé motif "forbidden". Critère bloquant : ≥ 95 pour qu'un modèle soit utilisable en production.
2. **Pertinence** — pourcentage de réponses contenant au moins 60% des `expected_keywords`.
3. **Format** — pourcentage respectant les contraintes de longueur et le ton.
4. **Coût** — coût total normalisé (le moins cher = 100, le plus cher = 0).

Score composite = $0.5 \times \text{justesse} + 0.25 \times \text{pertinence} + 0.15 \times \text{format} + 0.10 \times \text{coût}$.

Évaluation automatique vs humaine

Phase 1 — éval auto (dans le script) :

- Recherche de mots-clés "forbidden" par regex sur la liste fermée de motifs.
- Recherche de présence des `expected_keywords`.
- Mesure de longueur (mots, caractères).

Phase 2 — éval humaine :

- Tu (le mainteneur) évalues à la main 30 sorties par modèle dans un notebook Jupyter `pedagogy/scripts/eval_notebook.ipynb`.
- 3 critères, score 1–5 : pédagogie, exactitude perçue, naturel du français.
- Le notebook calcule l'accord inter-modèle.

Phase 3 — éval LLM-as-judge (optionnel) :

- Pour automatiser, demander à un LLM costaud (claude-opus-4-7 ou gpt-5.4) d'évaluer les sorties des modèles testés.
- Prompt de juge fourni dans `pedagogy/scripts/judge_prompt.py`.
- À utiliser uniquement après calibration avec l'éval humaine sur 30 cas.

Router de modèles (résultat final de la PR)

À l'issue du bench, configurer un **router** dans `claude_writer.py` (qui devient `llm_writer.py`) :

```
# pedagogy/explanations/llm_writer.py

MODEL_ROUTING_POLICY = {
    "default": os.getenv("PEDAGOGY_LLM_MODEL", "claude-haiku-4-5"),
    "high_severity_blunder": "claude-sonnet-4-6",      # On paie plus si la gaffe es
    "brilliant": "claude-opus-4-7",                    # Et pour célébrer un beau c
    "no_motif": "deepseek-v3.2",                       # Cas simple, modèle bon mar
}

def select_model(verdict: MoveVerdict) -> str:
    if any(m.role == "played" and m.motif == "sacrifice" for m in verdict.motifs):
        if verdict.verdict == Verdict.BRILLIANT:
            return MODEL_ROUTING_POLICY["brilliant"]
    if verdict.verdict == Verdict.BLUNDER and verdict.delta_winchance > 0.5:
        return MODEL_ROUTING_POLICY["high_severity_blunder"]
    if not verdict.motifs:
        return MODEL_ROUTING_POLICY["no_motif"]
    return MODEL_ROUTING_POLICY["default"]
```

Critères d'acceptation de la PR

- Au moins 5 modèles testés sur le dataset de 30 cas.
- Tableau récapitulatif dans `pedagogy/BENCH_RESULTS.md` avec coût, latence, scores.
- Le modèle choisi par défaut atteint **justesse ≥ 95** et **score composite ≥ 70** .
- Coût moyen par partie analysée affiché dans le README.
- Tests unitaires de chaque `LLMClient` avec mocks HTTP.
- Documentation de comment ajouter un nouveau provider.

Livraison additionnel — rapport `BENCH_RESULTS.md`

```
# Bench LLM — Writer pédagogique

Date : 2026-XX-XX
Dataset : 30 cas couvrant 6 catégories
Évaluation : auto (anti-halluc + keywords) + humaine (50 sorties annotées)

## Résultats

| Modèle | Justesse | Pertinence | Format | Coût total | Latence p50 | Score |
| --- | --- | --- | --- | --- | --- | --- |
| claude-opus-4-7 | 100 | 95 | 98 | $0.84 | 1850ms | 78 |
| claude-sonnet-4-6 | 98 | 92 | 96 | $0.50 | 1200ms | 82 |
| claude-haiku-4-5 | 96 | 88 | 95 | $0.04 | 600ms | **91** |
| gpt-4.1-nano | 88 | 70 | 92 | $0.01 | 450ms | 65 (justesse insuffisante) |
| deepseek-v3.2 | 94 | 82 | 88 | $0.01 | 800ms | 84 |
| gemini-2.5-flash | 93 | 84 | 90 | $0.08 | 700ms | 83 |

**Conclusion** : claude-haiku-4-5 est le meilleur compromis avec une économie de

## Recommandation de routage

- 90% du trafic → claude-haiku-4-5
- 8% (blunder élevés) → claude-sonnet-4-6
- 2% (brillant + sacrifice rare) → claude-opus-4-7

Coût estimé sur 1000 parties/mois : ~6 € (vs 720 € en tout-opus).
```

19. Intégration dans l'application Draught Master

Contexte d'intégration

Le framework `pedagogy/` est un **module backend** du repo `ai-draught/` lui-même. Il n'est pas un package distribué séparément (pour l'instant). L'application Draught Master consomme `pedagogy/` :

1. **Backend** : via imports Python directs depuis `main.py` (déjà couvert par §9 et PR 8).
2. **Frontend** : via les endpoints HTTP `/api/pedagogy/*` exposés par le backend.

Aucune publication de package PyPI/npm n'est requise en v1. Si le besoin émerge (autre app voulant réutiliser le framework), il faudra une PR séparée pour extraire en package

autonome — couvert au §19.5.

19.1 Points de greffe dans le frontend existant

Trois composants existants doivent être enrichis pour consommer la nouvelle API. **Aucun ne doit être réécrit** : on ajoute des sections / panneaux complémentaires.

a) `AnalysisPanel.tsx`

Composant déjà chargé de l'analyse Claude libre. On y ajoute un onglet "Verdict pédagogique" qui consomme `/api/pedagogy/move-verdict/{game_id}/{move_number}`.

Maquette :

[Analyse Claude] [Verdict pédagogique]	← onglets
Coup 17. 32-28 (Blancs)	
Verdict : Δ ERREUR (-0.42 → +0.18)	
Phase : milieu de jeu	
Motifs détectés :	
• Coup royal manqué (sévérité 1.0)	
↳ La séquence 40-34, 34x14, 14x3 capturerait 7 pions.	
Commentaire :	
Vous avez raté un coup royal ! La séquence 40x3 aurait capturé 7 pions. Surveillez toujours ces configurations centrales.	
📖 Référence : Dubois, ch. 12, p. 87	
[Voir le diagramme similaire →]	

Le board doit également **highlighter les cases impliquées** dans le motif

(`MotifMatch.squares`) via une nouvelle prop `highlightedSquares: { color: string; squares: number[] }[]` ajoutée à `Board.tsx`.

b) `GameHistory.tsx`

Ajouter une colonne "Précision pédagogique" calculée depuis le profil agrégé, et un filtre par motif rencontré ("Mes parties où j'ai subi un coup royal").

c) `UserStatsCard.tsx`

Nouvelle section "Forces & faiblesses" alimentée par `/api/pedagogy/profile/{user_id}` :

Forces	Faiblesses
✓ Défense (12 parties)	✗ Coup royal subi (7/30 parties)
✓ Finales dame (8/10)	✗ Centre abandonné (12/30 parties)
	✗ Prise max ratée (4/30 parties)

Phase la plus faible : milieu de jeu
Exercices recommandés : [Voir →]

19.2 Nouveaux composants à créer

- `PedagogyTab.tsx` : panneau onglet à l'intérieur de `AnalysisPanel.tsx`.
- `MotifBadge.tsx` : badge coloré par type de motif (vert = joué, rouge = manqué, orange = subi).
- `MotifTooltip.tsx` : tooltip explicatif au survol d'un motif, avec lien vers le livre source.
- `WeaknessChart.tsx` : graphique radar des faiblesses du joueur (recharts).
- `RecommendedExercisesPanel.tsx` : liste d'exercices recommandés depuis `/api/pedagogy/profile/me/recommendations`.

19.3 Évolutions des types TypeScript

Ajouter à `frontend/src/types.ts` :

```
export type MotifRole = "played" | "missed" | "suffered" | "threatened";

export interface MotifMatch {
  motif: string;
  role: MotifRole;
  squares: number[];
  pv: string[];
  severity: number;
  metadata: Record<string, unknown>;
}
```

```

export type Verdict =
  | "brilliant" | "best" | "excellent" | "good"
  | "inaccuracy" | "mistake" | "blunder"
  | "forced" | "book";

export type Phase = "opening" | "middlegame" | "endgame";

export interface MoveVerdict {
  move_number: number;
  side: "white" | "black";
  move_notation: string;
  fen_before: string;
  fen_after: string;
  score_before: number;
  score_after: number;
  delta_winchance: number;
  verdict: Verdict;
  is_forced: boolean;
  motifs: MotifMatch[];
  phase: Phase;
  explanation?: string;
  explanation_mode?: "template" | "template+book" | "llm";
}

export interface UserProfile {
  user_id: number;
  games_count: number;
  average_accuracy: number;
  strengths: { motif: string; frequency: number; played: number }[];
  weaknesses: { motif: string; frequency: number; missed: number; suffered: number }[];
  weakest_phase: Phase;
  recommended_exercise_tags: string[];
}

```

19.4 Évolution du client API

Ajouter à `frontend/src/api/client.ts` :

```

export const pedagogyApi = {
  analyzeGame: (gameId: number, mode: "fast" | "deep" = "fast") =>
    http.post<{ verdicts: MoveVerdict[]; summary: object }>(
      `/api/pedagogy/analyze-game`, { game_id: gameId, mode }
    ),

  getMoveVerdict: (gameId: number, moveNumber: number) =>
    http.get<MoveVerdict>(`/api/pedagogy/move-verdict/${gameId}/${moveNumber}`),

```



```

explainMove: (gameId: number, moveNumber: number,
              mode: "template" | "template+book" | "llm" = "template",
              lang: "fr" | "en" = "fr") =>
  http.post<{ explanation: string }>(`/api/pedagogy/explain-move`,
    { game_id: gameId, move_number: moveNumber, mode, lang }),

getProfile: (userId: number) =>
  http.get<UserProfile>(`/api/pedagogy/profile/${userId}`),

getRecommendations: (n = 10) =>
  http.get<Exercise[]>(`/api/pedagogy/profile/me/recommendations?n=${n}`),
};

```

19.5 Cas futur — extraction en package autonome

Si le besoin émerge :

- **Renommer** `backend/pedagogy/` en `draught-pedagogy/` au niveau racine.
- **Publier** sur PyPI sous `draught-pedagogy`.
- **Découpler** des modules `db/` et `scan_engine/` du repo principal — le package ne dépendrait plus que de `game_engine.py` (qui pourrait lui aussi être extrait en `draught-rules`).
- **Définir** une interface `EngineProtocol` permettant à n'importe quel moteur (Scan, KingsRow, etc.) d'être branché.

Cette extraction est **hors scope** de cette spec. Mentionné ici pour orienter les choix de conception en v1 :

- Aucune dépendance directe au schéma SQLite spécifique de Draught Master dans les modules `features/` et `motifs/`.
- Tous les accès DB passent par `pedagogy/storage.py` (couche d'abstraction).
- Aucun import depuis `db/` dans `features/`, `motifs/`, `verdicts/`, `explanations/` — uniquement dans `storage.py` et `api.py`.

19.6 Critères d'intégration

L'intégration est considérée comme réussie quand :

- Un utilisateur joue une partie, va dans son historique, clique sur "Analyser", et voit pour chaque coup le verdict + motif + commentaire en moins de 30 secondes.

- Le board highlight visuellement les cases impliquées dans le motif détecté.
 - La page profil affiche des forces et faiblesses agrégées sur les 30 dernières parties.
 - Cliquer sur "Exercices recommandés" amène vers des exercices effectivement liés aux faiblesses détectées.
 - Le composant `AnalysisPanel.tsx` continue de fonctionner exactement comme avant pour les utilisateurs qui n'utilisent que l'onglet Claude libre.
 - Le i18n existant (FR/EN) couvre toutes les nouvelles chaînes.
-

20. Annexe — Exemple complet exécuté

Cette annexe montre **un demi-coup réel** traversant tout le pipeline, depuis la position FEN jusqu'au commentaire en français, pour que l'agent comprenne le ton et le niveau de détail attendus.

Contexte

Partie : Sijbrands vs Sheroeatan, début des années 70 (référence Dubois p. 4, chapitre "combinaisons d'ouverture"). Demi-coup à analyser : coup **24** des Blancs, position légèrement risquée pour les Noirs.

Entrée

```
{
  "game_id": 12345,
  "move_number": 24,
  "fen_before": "W:W26,27,28,32,33,34,37,38,39,40,41,42,43,46,47,48,49,50:B1,2,3,
  "fen_after": "...",
  "move_played": "27-21",
  "scan_score_before": 0.18,
  "scan_score_after": -0.65,
  "scan_best_move": "34-30",
  "scan_best_pv": ["34-30", "25x34", "39x10", "5x14", "40-34"]
}
```

Étape 1 — `compute_features(state_before)`

```
Features(
  white_men=18, white_kings=0,
  black_men=18, black_kings=0,
```

```

material_balance=0,
center_count_white=4,          # 27, 28, 32, 33
center_count_black=3,         # 18, 19, 23
left_wing_white=2, right_wing_white=3,
left_wing_black=1, right_wing_black=2,
isolated_pawns_white=[],
isolated_pawns_black=[23],    # pion 23 isolé
backward_pawns_white=[46, 47, 48, 49, 50],
backward_pawns_black=[1, 2, 3, 4, 5],
holes_white=[],
holes_black=[24],             # case 24 vide à l'intérieur de la zone noire
outposts_white=[27, 28],
outposts_black=[],
white_legal_moves=11,
black_legal_moves=9,
white_promotion_distance=8,
black_promotion_distance=8,
formations=["classique_blancs"],
phase=Phase.MIDDLEGAME,
)

```

Étape 2 — détection des motifs

Pour le coup joué 27-21 (qui sacrifie un pion sur l'aile gauche sans compensation immédiate) :

```

# detect_all(state_before, move=27-21, state_after, pv=[...], scores=(0.18, -0.65

motifs_played = []
# - CoupRoyalDetector : len(captures)=0, retourne None
# - CoupTurcDetector : pas de captures, retourne None
# - CoupDeTalonDetector : pas de captures, retourne None
# - EnvoiADameDetector : pas de promotion forcée détectée, retourne None
# - SacrificeDetector : material_loss = 1, mais score_after = -0.65 (mauvais pour
#                         retourne None (mauvais sacrifice = pas un "sacrifice tacti
# - PriseMaxRateeDetector : pas de capture jouée, mais aucune capture obligatoire
# - PerceeDetector : pas de pion à 2 coups de promotion, retourne None

motifs_missed = []
# Examen de scan_best_move=34-30, scan_best_pv=[34-30, 25x34, 39x10, 5x14, 40-34]
# - CoupRoyalDetector.detect_missed : la PV contient une rafle 39x10 + 40-34 qui.
#   captures dans 39x10 = [25, 14, 4] = 3 captures → pas assez pour coup royal
#   retourne None
# - SacrificeDetector.detect_missed : le coup best implique un sacrifice 34-30 pu
#   bal après séquence complète = +2 pour blanc, score_pv_end ≈ +1.2
#   → MotifMatch(motif="sacrifice", role="missed", squares=[34, 30, 25, 39, 10,

```

```
#          pv=["34-30", "25x34", "39x10", "5x14", "40-34"], severity=0.75
#          metadata={"material_gain_after_pv": 2, "pv_length": 5})

motifs = motifs_played + motifs_missed
# = [MotifMatch(motif="sacrifice", role="missed", ...)]
```

Étape 3 — assemblage du verdict

```
# classify_move(score_before=0.18, score_after=-0.65, side="white", is_forced=False)
# wc_before = +1 * winchance(0.18) = +0.178
# wc_after = +1 * winchance(-0.65) = -0.567
# delta = 0.178 - (-0.567) = 0.745 → BLUNDER

MoveVerdict(
    move_number=24,
    side="white",
    move_notation="27-21",
    fen_before="W:W26,27,28,32,33,34,37,38,39,40,41,42,43,46,47,48,49,50:B1,2,3,4",
    fen_after="...",
    score_before=0.18,
    score_after=-0.65,
    delta_winchance=0.745,
    verdict=Verdict.BLUNDER,
    is_forced=False,
    motifs=[MotifMatch(motif="sacrifice", role="missed", severity=0.75, ...)],
    phase=Phase.MIDDLEGAME,
)
```

Étape 4 — explication mode `template`

```
# Lookup TEMPLATES_FR[("sacrifice", "missed", Verdict.BLUNDER)]
# Pas trouvé. Fallback sur TEMPLATES_FR[("sacrifice", "missed", None)] :

template = (
    "Vous avez raté un sacrifice gagnant ! La séquence {pv} aurait gagné "
    "{material_gain_after_pv} pions. Repérer les sacrifices reste l'une des "
    "compétences les plus précieuses du milieu de jeu."
)

# Rendu avec metadata du MotifMatch :
"Vous avez raté un sacrifice gagnant ! La séquence 34-30, 25x34, 39x10, 5x14, 40-
"aurait gagné 2 pions. Repérer les sacrifices reste l'une des compétences les plu
"précieuses du milieu de jeu."
```

Étape 5 — explication mode `template+book`

```
# BookRAG.search("sacrifice", max_results=1) →
# [{"book": "Dubois Perfectionnement", "page": 64,
#   "excerpt": "Le sacrifice de pion en milieu de jeu vise à...",
#   "score": 0.82}]

# Résultat :
"Vous avez raté un sacrifice gagnant ! La séquence 34-30, 25x34, 39x10, 5x14, 40-
"aurait gagné 2 pions. Repérer les sacrifices reste l'une des compétences les plu
"précieuses du milieu de jeu.\n\n"
"Pour approfondir : Dubois Perfectionnement p.64."
```

Étape 6 — explication mode `llm` (avec Claude Haiku après bench)

Prompt envoyé au LLM :

```
SYSTEM:
Tu es un commentateur pédagogique pour le jeu de dames international.
[... cf. $7 SYSTEM_PROMPT ...]

USER:
COUP JOUÉ: 27-21 (white)
VERDICT: blunder
SCORES SCAN: +0.18 -> -0.65

MOTIFS DÉTECTÉS:
- sacrifice (missed) - sévérité 0.75 - métadonnées {'material_gain_after_pv': 2,
  'pv_length': 5, 'pv': ['34-30', '25x34', '39x10', '5x14', '40-34']}

POSITION:
Centre blancs/noirs: 4/3, Matériel: +0, Formations: ['classique_blancs'], Phase:

EXTRAITS PÉDAGOGIQUES PERTINENTS:
> Le sacrifice de pion en milieu de jeu vise à obtenir une supériorité de mobilit
> ou de tempo sur l'aile opposée. Il s'oppose au principe de conservation matéri
> mais constitue souvent le seul moyen de briser une formation classique.

Rédige le commentaire.
```

Sortie attendue :

Vous avez manqué une jolie opportunité. La séquence 34-30 amorce un sacrifice qui se solde par un gain de 2 pions après la rafle 39x10 puis 40-34. Avec votre formation classique solide, ce type de sacrifice tactique est exactement la

ressource qu'il faut envisager – Dubois en discute au chapitre sur la rupture de la formation classique.

Vérification anti-hallucination : le mot “sacrifice” est dans la liste des motifs détectés. Aucun mot-clé interdit (coup royal , coup turc , coup de talon , envoi à dame , percée , etc.) n’apparaît. ✓ Acceptée.

Étape 7 — persistance

```
INSERT INTO move_verdicts (game_id, move_number, ..., motifs_json) VALUES (
    12345, 24, ...,
    '[{"motif": "sacrifice", "role": "missed", "severity": 0.75, ...}]'
);

INSERT INTO pedagogy_explanations (move_verdict_id, mode, lang, text) VALUES (
    <id>, 'llm', 'fr', 'Vous avez manqué une jolie opportunité. ...'
);
```

Étape 8 — réponse API

GET /api/pedagogy/move-verdict/12345/24 retourne :

```
{
  "move_number": 24,
  "side": "white",
  "move_notation": "27-21",
  "fen_before": "W:W26,27,28,...",
  "score_before": 0.18,
  "score_after": -0.65,
  "delta_winchance": 0.745,
  "verdict": "blunder",
  "is_forced": false,
  "phase": "middlegame",
  "motifs": [
    {
      "motif": "sacrifice",
      "role": "missed",
      "squares": [34, 30, 25, 39, 10, 5, 14, 40],
      "pv": ["34-30", "25x34", "39x10", "5x14", "40-34"],
      "severity": 0.75,
      "metadata": {"material_gain_after_pv": 2, "pv_length": 5}
    }
  ],
  "explanation": "Vous avez manqué une jolie opportunité. La séquence 34-30 amorc",
  "explanation_mode": "llm"
}
```

}

Étape 9 — rendu frontend (cf. §19.1)

Le board highlight les cases [34, 30, 25, 39, 10, 5, 14, 40] en orange. Le panneau verdict affiche le commentaire, le badge "Sacrifice manqué" (rouge), et un bouton "Voir la séquence" qui rejoue la PV coup par coup sur le damier.

Étape 10 — agrégation dans le profil utilisateur

Après cette partie + 29 autres, l'aggregator détecte que ce joueur a `sacrifice missed` dans 8/30 parties → entre dans les faiblesses du profil → des exercices Dubois taggés `sacrifice` lui sont recommandés.

21. Annexe — Squelette de table de motifs étendue

Pour faciliter l'extension future, voici une liste exhaustive des motifs à terme ($\geq v3$) :

Tactiques (atomiques, détectables sur 1-2 demi-coups)

Motif	Détection	Priorité
Coup royal	rafle ≥ 6 pions au centre	P1
Coup turc	trajectoire repassant par même case	P1
Coup de talon	renversement de direction dans rafle	P1
Coup Philippe	sacrifice aile + rafle	P2
Coup Raphaël	sacrifice 27/24 + rafle longue	P2
Coup de l'express	rafle ≥ 5 en ligne droite	P2
Coup Bonnard	sacrifice + dame adverse capturée immédiatement	P2
Coup Fabre	cf. Dubois ch. 21	P3
Coup de mazette	combinaison élémentaire 2-3 pions	P3
Coup Manoury	cf. littérature néerlandaise	P3
Coup du cheval	dégagement puis rafle en L	P3

Coup Springer	rafle d'ouverture sur grande diagonale	P3
---------------	--	----

Tactiques composées (multi-coups, détectables sur 3-5 demi-coups)

Motif	Détection	Priorité
Sacrifice gagnant	matériel -1, score +0.5 dans PV	P1
Envoi à dame	promotion forcée ≤ 2 coups	P1
Percée	pion à 2 coups de promotion non interceptable	P2
Enchaînement Weiss	clouage spécifique avec cases 27/22	P2
Damnation	dame adverse coincée et capturée	P3
Pat tactique	adversaire sans coup légal	P3

Stratégiques (sur la position, sans rafle)

Motif	Détection	Priorité
Centre fort/faible	center_count ≥ 5 ou ≤ 2	P1
Pion arrière isolé	backward + isolated	P1
Avant-poste	outpost détecté	P2
Trou	hole détecté	P2
Formation classique	match KNOWN_FORMATIONS	P1
Formation Roozenburg	match KNOWN_FORMATIONS	P2
Formation Ghestem	match KNOWN_FORMATIONS	P2
Tempo perdu	diff. développement > 2	P3
Aile débordée	pions adverses $>$ propres sur une aile, +2	P3

Finales (≤ 8 pièces totales)

Motif	Détection	Priorité

Opposition dame-pion	finale type avec opposition	P2
Dame contre 3 pions	finale gagnante connue	P2
Coup de l'escalier	technique de finition	P3
Triangle de Petrov	configuration dame-roi	P3
Position-clé Weiss	finale-clé canonique	P3

L'agent doit implémenter en priorité P1, puis P2 dans une PR ultérieure (PR 14 et PR 15+).

Fin de spécification. L'agent doit poser des questions sur ce document avant de commencer, plutôt que de présumer. Les §18, 19 et 20 sont essentiels pour l'usage final : ils encadrent respectivement l'optimisation des coûts LLM, l'intégration concrète dans Draught Master, et le ton/format attendu du livrable pédagogique.